



BugIt

by Taskivator

Manual completo, passo a passo

BugIt

Agente de QA para Registro de Bugs no VS Code

Configuração inicial e uso diário: um guia detalhado, clique a clique, escrito para quem nunca fez nada parecido antes. Não é preciso conhecimento técnico. Siga na ordem e você estará registrando tickets de bug impecáveis ainda hoje.

Funciona no: VS Code no Windows (principal), macOS & Linux

Aviso sobre a tradução.

Este documento foi traduzido automaticamente e não passou por revisão de falantes nativos. A versão em inglês é a que prevalece: em caso de divergência, vale o texto em inglês. Para a redação mais precisa e atual, consulte o documento em inglês.

O QUE VOCÊ VAI ENCONTRAR AQUI

Faça a **Parte 1** uma vez, na ordem. Tudo depois disso você pode consultar sempre que precisar. A **Solução de problemas** e o **FAQ** perto do final respondem às dúvidas mais comuns, então confira-os antes de mandar um e-mail para o suporte.

Parte 1 · Configuração (faça isso uma vez)

- Passo 0: Obtenha o GitHub Copilot
- Passo 1: Instale o VS Code
- Passo 2: Descompacte seu download do BugIt
- Passo 3: Abra o BugIt no VS Code
- Passo 4: Escolha o agente BugIt no chat
- Passo 5: Ative sua licença
- Passo 6: Aceite os termos (uma vez)

Passo 7: Rode "Begin setup"

Passo 8: Conecte seu rastreador

Passo 9: Salve seus logins

Passo 10: Confirme que está tudo pronto

Parte 2 · Uso diário

Parte 3 · Deixe do seu jeito

Parte 4 · Atualizações, backups e segurança

Parte 5 · Solução de problemas e FAQ

Parte 6 · Licença e suporte

A única regra que mantém você seguro

O BugIt **nunca** registra nada sem antes te mostrar um preview e você digitar `FILE IT`, e um simples "sim" não basta para uma ação irreversível. Você está sempre no controle. Quer praticar com risco zero? Digite `dry run` e ele escreve o relatório inteiro **sem** registrá-lo: o dry-run suprime todo ticket, comentário, edição, anexo, remoção e notificação externos (comandos locais que você mesmo roda, como backup ou exportação, ainda podem criar arquivos locais) e instrui o agente a recusar gravações no rastreador (essa recusa segue as instruções do BugIt, não é um bloqueio de plataforma); para avaliar, use credenciais somente leitura.

Configuração inicial x uso diário

Você só segue toda a **Parte 1** uma vez. Depois disso, abrir o BugIt é rápido, e as atualizações são um único comando (`Update`). Veja a Parte 4.

PARTE 1 · CONFIGURAÇÃO: FAÇA ISSO UMA VEZ

Cerca de 15 minutos. Siga os passos na ordem; cada um depende do anterior. Não pule etapas.

Obtenha o GitHub Copilot

Este guia segue o caminho do GitHub Copilot

O BugIt é o "motorista" e a IA é o "motor" (é ela que realmente escreve os relatórios). Estes passos configuram você com o GitHub Copilot no VS Code, o caminho mais rápido se você está começando do zero. O BugIt também funciona com a extensão do Claude, com qualquer outro assistente capaz de executar um comando e sozinho com a sua própria chave de IA. Se você já usa um deles, pule para o Passo 2 e abra a pasta do BugIt nessa ferramenta em vez do VS Code.

O que é: o GitHub Copilot é um assistente de IA da GitHub (que pertence à Microsoft). Ele tem um nível gratuito para uso leve e um nível pago para uso intenso. Você vai instalá-lo no Passo 1 e fazer login.

1. Você vai precisar de uma **conta do GitHub**. Se não tiver uma, crie uma gratuita em github.com/signup.
2. O nível gratuito do Copilot já é suficiente para experimentar o BugIt. Se você registra bugs o dia todo, vale a pena o plano pago "Copilot Pro", e você pode fazer upgrade a qualquer momento em github.com/features/copilot.
3. Você vai realmente ativar o Copilot dentro do VS Code no Passo 1, então não precisa fazer mais nada aqui por enquanto.

Instale o VS Code

O VS Code é um aplicativo gratuito da Microsoft. É a "janela" onde o BugIt vive.

1. Abra seu navegador e acesse code.visualstudio.com.
2. Clique no grande botão azul **Download** (ele detecta seu sistema automaticamente).
3. Abra o arquivo baixado e instale-o: **aceite todas as opções padrão**, apenas clique em Next/Install até o fim.
4. Abra o VS Code assim que a instalação terminar.

Ative o GitHub Copilot dentro do VS Code

1. Na extrema esquerda do VS Code, clique no ícone de **Extensions** (parece quatro quadradinhos).
2. Na caixa de busca, digite **GitHub Copilot**.
3. Clique em **Install** em "GitHub Copilot" (e, se for oferecido, em "GitHub Copilot Chat").
4. Um aviso vai pedir para você **Sign in to GitHub**. Clique nele e faça login com sua conta do GitHub na janela do navegador que abrir.

✓ **Você vai saber que funcionou quando:** um pequeno ícone do Copilot aparecer na parte de baixo do VS Code, e o painel de Chat abrir (o ícone de balão de fala à esquerda, ou pressione **Ctrl+Alt+I**).

2

Descompacte seu download do BugIt

Sua compra veio com um arquivo .zip (por exemplo, `bugit-qa-agent.zip`). Você precisa descompactá-lo; rodá-lo ainda zipado não funciona.

1. Encontre o arquivo .zip (geralmente na sua pasta **Downloads**).
2. **Clique com o botão direito** nele → escolha **Extract All...** (Windows) ou dê um duplo clique nele (Mac).
3. Escolha um local simples, como sua pasta **Documents**, e clique em Extract.
4. Agora você terá uma pasta **BugIt** com muitos arquivos dentro. Lembre-se de onde ela está.

Não coloque na OneDrive nem em uma pasta sincronizada

Pastas de sincronização na nuvem podem causar conflitos de arquivo. Sua pasta Documents ou a Área de Trabalho é perfeita. Também não renomeie os arquivos internos; deixe a pasta como está.

3

Abra o BugIt no VS Code

1. No VS Code, clique em **File → Open Folder** (menu superior esquerdo).
2. Navegue até onde você descompactou o BugIt (por exemplo, Documents).
3. Clique uma vez na pasta **BugIt** para selecioná-la, depois clique em **Select Folder**.
4. Se o VS Code perguntar "Do you trust the authors of the files in this folder?", clique em **Yes, I trust the authors**.

✓ **Você vai saber que funcionou quando:** você vir os arquivos do BugIt (pastas como `tools`, `.github`, e arquivos como `README.md`) listados na lateral esquerda do VS Code.

4

Escolha o agente BugIt no chat

1. Abra o painel de **Chat** do Copilot: clique no ícone de balão de fala à esquerda, ou pressione `Ctrl+Alt+I` (Windows) / `Cmd+Alt+I` (Mac).
2. No topo da caixa de chat há um pequeno **menu suspenso de modo** (pode estar escrito "Ask" ou "Agent").
3. Clique nele e escolha **BugIt QA Agent** na lista.

Não está vendo "BugIt QA Agent" na lista?

Confira se você abriu a pasta do BugIt (Passo 3), e não outra pasta; o agente carrega de dentro dela. Feche e reabra a pasta se for preciso.

5

Ative sua licença

Tudo o que você precisa é uma conta Buglt; nada de código de licença para copiar ou colar. A ativação acontece no seu navegador.

1. Na caixa de chat, digite **hi** e pressione Enter.
2. Depois digite **Activate**; o Buglt abre o Buglt Portal no seu navegador. Entre na sua própria conta, escolha o seu direito Solo ou Team e aprove este dispositivo; volte para o VS Code e a ativação termina automaticamente. Sua senha permanece no navegador e nunca é digitada no VS Code.

✓ **Você vai saber que funcionou quando:** o Buglt confirmar a ativação e informar a conta com que você entrou.

Mantenha sua conta privada

Sua conta e o seu direito estão vinculados a você. Não compartilhe, publique nem revenda; um acesso compartilhado pode ser revogado. Os membros da equipe (Team) entram cada um com a sua própria conta; não há código de licença compartilhado nem login compartilhado.

6

Aceite os termos (uma vez)

Na primeira vez, o Buglt mostra um resumo curto e pede que você aceite antes de fazer qualquer outra coisa.

1. Leia o resumo curto que ele mostra (você revisa cada ticket antes de ser registrado; a segurança do seu projeto é responsabilidade sua; as proteções são auxílios, não garantias, e, como elas são seguidas por qualquer modelo de IA que estiver respondendo por você no Copilot, um modelo mais leve/gratuito pode ocasionalmente pular um passo ou precisar que um comando seja repetido, algo que um modelo mais forte não faria).
2. Escolha **I agree** entre as duas opções que o Buglt mostra.

✓ **Você vai saber que funcionou quando:** o Buglt confirmar e seguir para a configuração. Isso só é perguntado uma vez; ele lembra.

7

Rode "Begin setup"

Agora o Buglt se configura sozinho, entrevistando você em linguagem simples. Você nunca edita um arquivo de configurações manualmente.

VOCÊ DIGITA
`Begin setup`

Ele vai fazer perguntas curtas e ativar somente o que você escolher. A mais importante é sobre o seu rastreador.

8

Conecte seu rastreador

A configuração registrou *qual* rastreador você usa. Este passo é o que permite ao BugIt registrar tickets nele de verdade: você cria um token de API na sua própria conta e o salva neste computador com um comando. O BugIt registra direto pela API REST do seu rastreador usando esse token, então nada precisa ficar rodando e o registro nunca usa um servidor MCP.

8a · Crie um token na sua própria conta

O SEU RASTREADOR	ONDE CRIAR
Jira Cloud	id.atlassian.com/manage-profile/security/api-tokens
Azure DevOps	dev.azure.com/YOUR-ORG/_usersSettings/tokens
GitHub Issues	github.com/settings/personal-access-tokens/new
GitLab Issues	gitlab.com/-/user_settings/personal_access_tokens
Bugzilla	no seu próprio Bugzilla: Preferences → API Keys
YouTrack	no seu próprio YouTrack: avatar → Profile → Account Security → Authentication
Linear	linear.app/settings/account/security
Shortcut	app.shortcut.com/settings/account/api-tokens
ClickUp	app.clickup.com/settings/apps
Asana	app.asana.com/0/my-apps
Trello	trello.com/apps/admin (uma chave de API E um token que você autoriza com ela)

Copie seu token assim que ele for exibido

A maioria dos serviços mostra um token novo apenas uma vez. Copie-o imediatamente e guarde em um lugar seguro até o BugIt salvá-lo para você (Passo 9). Escolha o prazo de validade mais longo que seu rastreador oferecer, para não precisar refazer isso tão cedo.

8b · Salve no seu computador

Abra um terminal dentro do VS Code e rode o comando do seu rastreador, usando `ado`, `github`, `gitlab` e assim por diante no lugar de `jira`.

VOCÊ RODA (NO TERMINAL)

```
python tools/connect.py jira
```

8c · Cole o token no prompt mascarado

O comando pede o token em um prompt local mascarado: ele nunca aparece na tela e nunca chega ao chat, a um ticket ou ao `config.json`. Cole ali e em nenhum outro lugar. O BugIt então faz login, lista os projetos que o token realmente enxerga, lê os tipos de item que você pode criar e **se recusa a salvar uma credencial que não consegue registrar**. Se a configuração termina, o registro funciona.

8d · Confira se está conectado

Para confirmar a qualquer momento, rode a verificação no terminal ou pergunte no chat:

VOCÊ RODA (NO TERMINAL)

```
python tools/connect.py jira --check
```

VOCÊ DIGITA

```
Check status
```

✓ **Você vai saber que funcionou quando:** o BugIt listar seu rastreador como conectado / saudável.

Nada precisa ficar rodando

Seu token de API fica no cofre de credenciais do seu sistema operacional, não no VS Code. Então não há nada para iniciar quando você reabrir o VS Code: o BugIt já pode registrar. Você pode conferir quando quiser com `python tools/connect.py jira --check`.

9

Salve seus logins

Você não precisa fazer nada de especial para isso: quando você salva o token (Passo 8c), o BugIt guarda suas credenciais com segurança no cofre de credenciais embutido do seu computador (Windows Credential Manager / macOS Keychain / Linux Secret Service), nunca em um arquivo comum e nunca enviadas a lugar nenhum. Da próxima vez, você não precisa colar o token de novo.

Quer conferir o que já está salvo? Digite:

VOCÊ DIGITA

```
Check saved credentials
```

O BugIt mostra **quais rastreadores têm uma credencial salva, nunca os valores** das credenciais. Ele nunca exhibe nem copia o valor de um token; se um login parar de funcionar, veja a Solução de problemas na Parte 5.

10 Confirme que está tudo pronto

Peça para o Buglt checar tudo mais uma vez antes de você registrar de verdade:

VOCÊ DIGITA

`Check readiness`

Ele lista qualquer coisa que ainda esteja faltando (geralmente só "conecte seu rastreador" ou "ative"). Quando estiver tudo verde, você verá "🟢 Setup complete."

Pronto, a configuração acabou, para sempre

Você não vai repetir a Parte 1. De agora em diante, abrir o Buglt é: abrir a pasta → digitar `hi`. Para mudar qualquer escolha depois, é só digitar `Begin setup` e dizer o que quer mudar.

Sem GitHub Copilot? Existe um assistente de terminal

Se você não pode usar o Copilot, abra o Terminal embutido (Terminal → New Terminal) e rode `python tools/setup_wizard.py`. Ele faz algumas perguntas e escreve suas configurações sem a IA. Você também pode rodar o Buglt de forma standalone com sua própria chave de IA; veja o FAQ.

PARTE 2 · USO DIÁRIO

Depois de configurado, você trabalha totalmente pelo chat. Fale naturalmente, ou use os exemplos exatos abaixo. Digite `help` a qualquer momento para ver a lista completa.

Escrever um relatório de bug

A função principal do Buglt. Descreva o bug em uma frase simples; ele escreve o ticket completo, preenche automaticamente a versão e a categoria, verifica duplicatas, mostra um preview e registra depois de você digitar `FILE IT`, porque um simples "sim" não basta para uma ação irreversível.

VOCÊ DIGITA

`Write a bug report: o app fecha sozinho toda vez que toco em Salvar no iPhone`

Ele redige um título, passos para reproduzir, resultado esperado vs. real, severidade e plataforma, e sempre define o próprio campo de prioridade/severidade do rastreador para combinar, não apenas um padrão genérico. Quer mudar algo antes de registrar? É só dizer, por exemplo "deixa a severidade alta" ou "adiciona que começou na última build."

Bugs rápidos (registro ágil)

Para formatos comuns (crash, error, missing, layout, slow, stuck, data, visual, audio, blocker), o formulário rápido pula algumas perguntas e preenche a categoria e a prioridade para você, e pula a busca por duplicatas para economizar a ida e volta. Use quando você já souber que o problema é novo; o preview diz isso, e um relatório padrão continua rodando a verificação completa de duplicatas.

VOCÊ DIGITA (EXEMPLOS)

Quick bug: crash na inicialização depois de uma atualização
Quick bug: layout botão sobrepõe o texto na tela de configurações
Quick bug: blocker não consigo passar da etapa de pagamento
Quick bug: slow o painel demora 20 segundos para carregar

Relatórios de bug de crash

Se você conectou uma ferramenta de crash (como o Sentry), o BugIt faz tudo acima além de relacionar seu relatório ao crash e buscar a stack trace para você.

VOCÊ DIGITA

Write a crash bug report: o jogo trava ao abrir o mapa

Comentários de verificação e encerramento

Depois de testar uma correção, o BugIt escreve um comentário organizado para postar no ticket. Ele escreve (e, opcionalmente, posta) o comentário, mas não muda o status do ticket. Você ainda fecha/resolve/reabre o ticket você mesmo no seu rastreador.

VOCÊ DIGITA	O QUE VOCÊ RECEBE
Close #123	Pergunta qual cenário (correção confirmada, fechado a pedido, funciona como projetado, reabrir...) e então escreve esse comentário.
Write a verification comment for the build you tested on Windows	Um comentário de "correção confirmada" com os detalhes da build preenchidos.
Write a reopen comment for the build you tested on Windows	Um comentário de "o problema ainda acontece" (depois você reabre o ticket você mesmo).

Tradução

Traduza relatórios ou termos entre os idiomas configurados, usando exatamente as palavras da sua equipe (do seu glossário; veja a Parte 3).

VOCÊ DIGITA (EXEMPLOS)

Translate to ja: o botão de salvar não faz nada na tela de perfil
Translate this bug report to English: [cole o texto]

Consulte informações (especificações e glossário)

Se você conectou especificações ou preencheu seu glossário, faça perguntas ao BugIt sobre o seu projeto.

VOCÊ DIGITA (EXEMPLOS)

O que é [seu termo]?
Me explique o fluxo de checkout
O que há de novo nas especificações?

Detecção de duplicatas

Isso acontece automaticamente em um relatório padrão, antes de qualquer coisa ser escrita: o BugIt pesquisa no seu rastreador por tickets correspondentes (mesmo os que estão escritos de um jeito um pouco diferente) e te avisa, para que você nunca registre o mesmo bug duas vezes. O formulário rápido pula a busca de propósito para economizar a ida e volta, e diz isso no preview, então você sempre sabe qual dos dois recebeu.

Recursos avançados (se você os ativou)

DIGITE ISTO	O QUE FAZ
Triage digest	Standup instantâneo: bugs abertos por área/severidade, os mais antigos sinalizados.
Export bug to file	Salva o relatório como Markdown/CSV/JSON em vez de registrá-lo (ótimo antes de configurar seu rastreador).
Undo last filing	Mostra a última gravação no log de auditoria. O BugIt não consegue excluir nem fechar um ticket; depois de um novo preview e do FILE IT ele só remove um comentário ou anexo criado por ele mesmo.
(automático)	Marcadores de SLA em bugs urgentes · campos específicos por categoria.

A lista completa de comandos

VOCÊ DIGITA

help

...mostra todo comando disponível dentro do agente, a qualquer momento.

Pratique com segurança quando quiser

Digite **dry run** e o BugIt escreve o relatório inteiro **sem** registrar nada, perfeito para aprender ou testar. O dry-run suprime todo ticket, comentário, edição, anexo, remoção e notificação externos (comandos locais que você mesmo roda, como backup ou exportação, ainda podem criar arquivos locais) e instrui o agente a recusar gravações no rastreador (uma recusa que segue as instruções do BugIt, não um bloqueio de plataforma); para avaliar, use credenciais somente leitura.

PARTE 3 · DEIXE DO SEU JEITO: ADICIONE O CONHECIMENTO DO SEU PROJETO

Assim que sai da caixa, o Buglt é um QA agent de estado em branco. Veja como ensinar a ele o seu projeto depois da compra. Sem programar: na maior parte, é só conversar com ele no chat. Tudo o que você adiciona fica no seu computador.

1 · As palavras da sua equipe (o glossário)

Isso deixa as traduções e o palpite de categoria precisos para o seu projeto.

1. Na lista de arquivos, abra `.github/glossary/terms.template.md`.
2. Preencha a tabela com as palavras da sua equipe: nomes de recursos, nomes de telas, nomes de itens, abreviações e (se você usa dois idiomas) suas traduções.
3. Salve o arquivo (`Ctrl+S`). Pronto: o Buglt usa exatamente esses termos a partir de agora.

Atalho

Ative o "glossary auto-seed" durante o `Begin setup` e o Buglt sugere termos a partir dos seus tickets existentes, e você só aprova cada um.

2 · O estilo dos seus bugs (estilo da casa)

Quer que os tickets combinem exatamente com o formato da sua equipe: estilo de título, rótulos de severidade, campos obrigatórios? Você não descreve; você **mostra**:

1. No chat, diga: "adapte ao estilo da minha equipe."
2. Cole **um screenshot** de um ticket existente do seu rastreador.
3. O Buglt estuda (localmente, com qualquer dado pessoal removido antes) e salva seu formato, seguindo-o em todo ticket futuro.

3 · Suas categorias, plataformas e campos

Basta dizer ao Buglt em linguagem simples e ele atualiza suas configurações para você:

VOCÊ DIGITA (EXEMPLOS)

```
Minhas categorias são Gameplay, UI, Áudio e Rede
A gente testa no Windows e no Xbox
```

4 · Suas próprias ferramentas (conexões personalizadas)

Usa uma ferramenta que não está na lista integrada? Conecte-a como se fosse uma integrada:

VOCÊ DIGITA

```
Add a custom MCP server
```

Dê um nome curto para ela e o endereço dela (precisa ser `https://`); o Buglt se conecta e verifica a saúde dela para você.

5 · Simplesmente corrija ao longo do caminho

O jeito mais fácil de todos: quando o Buglt redigir um ticket, corrija qualquer coisa que você não gostar: um rótulo, a redação, a severidade. Com "learn from your edits" ativado (recomendado), ele lembra da sua preferência e a aplica da próxima vez. O conhecimento do seu projeto se acumula naturalmente, e nada disso jamais sai do seu computador.

Tudo o que você adiciona é seu e fica local

Seu glossário, estilo da casa, categorias e preferências aprendidas ficam no seu computador, recebem backup antes de cada atualização e nunca são enviados a ninguém.

6 · Compartilhando sua configuração com sua equipe (licença Equipe)

Depois de configurar seu glossário, estilo da casa e escolhas de rastreador do jeito que você quer, dê ao resto da sua equipe exatamente a mesma configuração em um único comando, em vez de repetir os passos 1-3 em cada máquina:

VOCÊ RODA (UMA VEZ, DEPOIS QUE SUA CONFIGURAÇÃO ESTIVER DO JEITO QUE VOCÊ QUER)

```
python tools/share.py export
```

Isso escreve `config.share.zip`: suas escolhas de rastreador/crash/comunicação, glossário e estilo da casa, tudo empacotado junto, sem senhas nem tokens dentro. Envie para sua equipe.

CADA PESSOA DA EQUIPE RODA

```
python tools/share.py import config.share.zip
```

Eles recebem exatamente a sua terminologia, estilo de bug e configuração de rastreador imediatamente, sem redigitar categorias e sem colar de novo um screenshot para o estilo da casa. A licença e as configurações do dispositivo de cada um permanecem intactas.

Se a importação mudar para onde os dados são enviados

Se uma configuração compartilhada adicionar uma nova conexão personalizada ou endereço de notificação, o Buglt mostra exatamente o que mudou antes de qualquer outra coisa acontecer; nada é aplicado silenciosamente. Ele também tira um snapshot da configuração atual da pessoa primeiro, então [Restore my settings](#) desfaz a importação se não for o que ela esperava.

PARTE 4 · ATUALIZAÇÕES, BACKUPS E SEGURANÇA

Recebendo atualizações (um comando)

Quando uma versão mais nova estiver disponível, o BugIt menciona isso uma vez quando você diz `hi`. As atualizações de software estão incluídas enquanto sua licença do BugIt estiver ativa. Para instalar:

VOCÊ DIGITA

Update

1. O BugIt tira um backup novo dos seus arquivos primeiro.
2. Ele baixa a nova versão e verifica se ela é genuína e não foi adulterada (ela é assinada criptograficamente, então uma atualização falsa ou alterada é recusada).
3. Ele instala sem mexer nas suas configurações, glossário, estilo da casa, licença ou tokens.
4. Ele pede para você recarregar o VS Code (`Ctrl+Shift+P` → digite "Reload Window" → Enter). Inicie um chat novo e você já está na versão nova.

Nunca é preciso apagar pastas

Diferente da primeira instalação, as atualizações são no local. Você nunca apaga nem baixa nada de novo, e sua configuração é sempre preservada e salva em backup.

O que é seguro editar manualmente e o que não é

Seguro, e mantido em cada atualização: `config.json`, `.github/instructions/house-style.instructions.md` (veja a Parte 3; essa é a forma suportada de personalizar o comportamento), `.github/glossary/terms.template.md`, `.github/instructions/learned.instructions.md` e `.vscode/mcp.json`.

Não é seguro: não edite estes manualmente

O arquivo do agente (`.github/agents/bugit-qa-agent.agent.md`), qualquer outro arquivo em `.github/instructions/`, e tudo em `tools/` são o próprio produto, não suas configurações. Uma atualização os **sobrescreve silenciosamente**, e, diferente dos arquivos acima, **não existe backup para restaurá-los**. O BugIt verifica isso na inicialização e novamente pouco antes de instalar uma atualização, e avisa você se encontrar alguma mudança, mas a regra mais segura é simplesmente não editá-los.

Backups e restauração

VOCÊ DIGITA	O QUE FAZ
<code>Back up my settings</code>	Salva um snapshot local do seu config, glossário, estilo da casa, endpoints MCP e aprendizado; o seu direito de dispositivo assinado e os valores de credenciais são excluídos.
<code>List backups</code>	Mostra cada snapshot salvo com sua data.

Restore my settings

Reverte para um snapshot (e tira um snapshot do seu estado atual primeiro, então a própria restauração pode ser desfeita).

Automático antes de cada atualização

O Buglt sempre faz backup antes de instalar uma nova versão, então uma atualização nunca pode perder sua configuração. Se algo parecer errado, **Restore my settings** resolve.

Segurança: o que é protegido

✔ Nada sem o seu **FILE IT**

Todo ticket e comentário é mostrado em preview; qualquer coisa irreversível exige que você digite **FILE IT**, e um simples "sim" não basta.

⚠ Dry-run = zero gravações

Diga **dry run** e o seu rastreador não é contatado de forma alguma, leituras incluídas. Na prática, o dry-run suprime todo ticket, comentário, edição, anexo, remoção e notificação externos (comandos locais que você mesmo roda, como backup ou exportação, ainda podem criar arquivos locais) e não há busca, nem verificação de duplicatas, nem teste de conexão, porque cada um deles também alcança o seu quadro e destrava o seu token salvo. **QA_AGENT_DRY_RUN=1** no seu terminal é a chave que os comandos incluídos aplicam em código; dizer "dry run" no chat pede ao assistente que se contenha, o que é útil mas não é a mesma garantia.

🔑 Nenhum segredo em arquivos

Tokens ficam no cofre de credenciais do seu SO, nunca em um arquivo comum, nunca enviados para nós.

🗃 Roda na sua máquina

Ele conversa apenas com as ferramentas que você conecta e com o seu próprio modelo de IA.

Seus dados e privacidade

A única coisa que o Buglt envia para a Taskivator são dados de licença/atualização: o login na sua conta Buglt (no navegador, no Buglt Portal), um ID de dispositivo com hash de mão única, um identificador de instalação aleatório, o rótulo do seu dispositivo (nome do host) e o nome do seu sistema operacional, a versão do Buglt e o filtro de plano que você escolhe, material criptográfico de desafio de vida curta, e o direito Solo ou Team que você aprova para este dispositivo. Não há código de licença; sua senha permanece no navegador. Seus relatórios de bug, especificações, glossário, configurações e tokens nunca saem do seu computador. Sem telemetria, sem analytics, nunca. Os detalhes completos estão no arquivo **PRIVACY.md** na sua pasta do Buglt.

Riscos e avisos importantes: por favor, leia

A IA pode errar

O Buglt redige relatórios com um modelo de IA, que pode interpretar mal detalhes ou inventar algo. Sempre leia o preview antes de confirmar. Você está aprovando o que será registrado.

Ele registra no seu rastreador de verdade

Depois que você aprova, um ticket de verdade é criado. Praticando? Use `dry run` para escrever o relatório sem registrá-lo.

A segurança do seu projeto é sua

Como você usa o BugIt, e a segurança dos seus projetos, dados e rastreadores, são de sua responsabilidade. As proteções reduzem o risco, mas não substituem sua revisão.

O que o BugIt não faz

O BugIt registra em todos os onze rastreadores que nomeia, cada um com uma credencial que você cria na sua própria conta e que o BugIt confere contra o destino escolhido antes de salvá-la. O que chega a um campo de prioridade ordenável varia por rastreador, e GitHub, GitLab e Trello não têm nenhum, então ali a severidade é escrita no texto do ticket. Confluence, Sentry e Notion são somente leitura; todo o resto se conecta pelo seu próprio servidor MCP compatível, que você configura com a ajuda do BugIt. Ele usa a sua IA (Copilot ou a sua chave da OpenAI/Anthropic), não um modelo embutido; a ocultação de dados é o melhor esforço; e ele roda no VS Code, na extensão do Claude e em um terminal comum. Uma licença = um dispositivo (Solo) ou até 5 membros ou assentos de dispositivo (Team). Resultados e consistência também dependem de qual modelo de IA está respondendo: um modelo mais forte segue os passos de segurança com mais confiabilidade do que um muito leve ou gratuito.

PARTE 5 · SOLUÇÃO DE PROBLEMAS E FAQ

Quando algo der errado, descreva primeiro para o assistente em palavras simples. A maioria dos problemas de configuração e de uso diário pode ser diagnosticada direto no editor.

✦ Tente isso primeiro: peça para a IA corrigir

Sempre que algo der errado (durante a configuração ou no uso do dia a dia), a correção mais rápida quase sempre é **perguntar ao assistente direto no chat**. Descreva o que aconteceu em palavras simples (por exemplo, "a configuração parou no meio do caminho" ou "meu rastreador não conecta") e peça para ele corrigir. A IA consegue diagnosticar e resolver a maioria dos problemas na hora. Por favor, tente isso antes de nos mandar um e-mail; resolve a grande maioria dos problemas em segundos.

Aplicando uma correção da IA: o botão "Keep"

Se o assistente corrigir algo mudando um arquivo, o VS Code mostra uma pequena barra Keep / Undo. Se você pediu a correção e está satisfeito com ela, clique em **Keep**, e a mudança é aplicada só na sua própria cópia, no seu computador. Clique em **Undo** para descartá-la. Nada que você mantém com Keep é compartilhado com mais ninguém.

O QUE VOCÊ VÊ

O QUE FAZER

"Activate to start"	Digite <code>Activate</code> ; o BugIt Portal abre no seu navegador. Entre, escolha o seu direito Solo ou Team e aprove este dispositivo. Ainda não tem uma conta? Consiga uma na loja.
O navegador não abriu, ou a ativação não foi concluída	Digite <code>Activate</code> de novo para reabrir o BugIt Portal, depois termine de entrar e de aprovar este dispositivo. A ativação acontece inteiramente no navegador; não há nada para colar.
"No rastreador enabled"	Digite <code>Begin setup</code> e escolha seu rastreador.
O registro é recusado	Rode <code>python tools/connect.py jira --check</code> . Ele diz quem você é para o rastreador, o que o BugIt pode criar, e lista em <code>fixes</code> a solução exata.
Ele continua pedindo para você ativar	Digite <code>Activate</code> e conclua o login no navegador. O BugIt nunca exibe os valores de credenciais salvas. (Primeira vez? Veja o Passo 8c.)
Ele não registra um bug	Digite <code>Check readiness</code> ; ele lista exatamente o que está faltando (geralmente o token do rastreador ainda não foi salvo).
Algo parece quebrado depois de uma atualização	Digite <code>Restore my settings</code> para reverter.
O agente parece "fora do script"	Digite <code>Read and follow all your given instructions</code> , ou comece um chat novo. Isso pode acontecer com mais frequência em um modelo de IA leve/gratuito; um modelo mais forte no seletor de modelos do Copilot é mais consistente.
As respostas parecem genéricas / não específicas do projeto	Dê mais contexto, ou mostre um ticket existente durante o <code>Begin setup</code> para ele aprender seu estilo. Rode o <code>Begin setup</code> de novo para refinar.
Você quer mudar uma escolha da configuração	Digite <code>Begin setup</code> e diga o que quer mudar (por exemplo, "adicionar um rastreador" ou "começar de novo") e ele reabre só aquela parte. Se a configuração foi simplesmente interrompida no meio, um <code>Begin setup</code> sozinho retoma de onde parou em vez de recomeçar.

Verificações de saúde integradas

`Check status` (minhas ferramentas estão conectadas?) · `Check readiness` (o que falta antes de eu poder registrar?). Para uma verificação mais profunda, abra Terminal → New Terminal e rode `python tools/selftest.py` .

Perguntas frequentes

Preciso saber programar?

Não. Se você consegue instalar um aplicativo e digitar uma frase, você consegue usar o BugIt. Ele cuida das partes técnicas para você.

Ele vai registrar bugs sem eu saber?

Nunca. Todo ticket e comentário é mostrado em preview e exige que você digite `FILE IT` ; um simples "sim" não basta. Use `dry run` para praticar sem nenhuma gravação.

Eu preciso iniciar um servidor toda vez?

Não. O registro usa o token de API que você salvou com `python tools/connect.py` , então não há nada para iniciar. Um servidor MCP só é necessário para as integrações opcionais somente leitura, como Confluence, Sentry e Notion.

Posso usar em dois computadores?

Um dispositivo por vez no Solo (até 5 membros no Team, um por membro). Mudar para um dispositivo novo é tranquilo: ative o novo, e o antigo deixa de poder renovar de imediato. A vaga dele é liberada assim que o dispositivo antigo se reconecta (confirmando que abriu mão do acesso) ou sua licença offline de 72 horas expira, então não há nenhum período de espera fixo. Digite `License status` para verificar.

E se o servidor de licenciamento estiver fora do ar?

Depois de uma ativação online bem-sucedida, uma licença em cache permite continuar trabalhando offline por até 72 horas, então uma queda do servidor (ou apenas um fim de semana sem internet) não te interrompe dentro dessa janela. Passado esse prazo, é necessária uma verificação online para continuar.

Eu preciso do GitHub Copilot?

É o jeito mais fácil. Se você não tiver, rode `python tools/setup_wizard.py` no Terminal para configurar, e o Buglt pode rodar de forma standalone com sua própria chave OpenAI/Anthropic (`python standalone/run.py`).

Meus dados são privados?

Sim. A Taskivator recebe apenas dados de licença/atualização: o login na sua conta Buglt (no navegador, no Buglt Portal), um ID de dispositivo com hash de mão única e o direito Solo ou Team que você aprova para este dispositivo. O software Buglt não usa a medição do Google Ads nem envia telemetria do produto. Não há código de licença; sua senha permanece no navegador. Seus tickets em si vão apenas para o modelo de IA e o rastreador que você conectar, nunca para nós.

Posso editar os arquivos do Buglt?

Você deve personalizar as suas partes: seu glossário, estilo da casa e configurações (Parte 3). Só não desative o licenciamento/ativação. Se você encontrar um bug de verdade na própria ferramenta, mande um e-mail para o suporte para que ele seja corrigido para todo mundo na próxima atualização.

Com quais rastreadores de bugs ele funciona?

Com onze, e o Buglt registra em todos eles: Jira Cloud, Azure DevOps, GitHub Issues, GitLab Issues, Bugzilla, YouTrack, Linear, Shortcut, ClickUp, Asana e Trello. Cada um tem configuração guiada, cada um escreve com uma credencial que você cria na sua própria conta, e o Buglt a confere contra o seu destino antes de salvá-la. O que muda é quais extras chegam: o GitHub, o GitLab e o Bugzilla não têm upload de anexos de nenhum tipo, o ClickUp aceita um arquivo e não consegue removê-lo, e GitHub, GitLab e Trello não têm campo de prioridade, então ali a sua severidade é escrita no texto do ticket. Escolha o seu durante a configuração e troque quando quiser com `Begin setup` .

Ele vai pegar bugs duplicados antes de eu registrar?

Sim, em um relatório padrão. Antes de qualquer coisa ser escrita, ele pesquisa no seu rastreador por relatórios parecidos (mesmo os escritos de um jeito um pouco diferente) e mostra as correspondências, para você evitar abrir o mesmo bug duas vezes. Você ainda decide se quer seguir em frente. O formulário rápido pula essa busca de propósito, para economizar a ida e volta, e o preview te avisa.

Ele consegue escrever tickets em mais de um idioma?

Sim. Adicione um segundo idioma durante a configuração e todo relatório é escrito em ambos, mantidos em blocos separados, usando seu glossário para os termos do produto ficarem exatos; ele nunca improvisa uma tradução.

Posso anexar um screenshot?

Peça ao BugIt para anexá-los. Ele cria uma pasta de evidências, abre a pasta e informa o caminho exato; coloque seus arquivos lá e avise que terminou. O BugIt então remove qualquer coisa sensível que perceber, mostra uma prévia e os anexa após o seu próprio [FILE IT](#). O BugIt não consegue alcançar um arquivo colado no chat, por isso sempre oferece a pasta.

Ele esconde senhas e tokens nos meus relatórios?

Quando a remoção de dados sensíveis está ativada, ele limpa e-mails, tokens e endereços IP de todo rascunho antes de registrar. Você sempre vê o texto completo no seu preview primeiro.

Registrei um ticket por engano. Dá para desfazer?

O preview e a confirmação digitada ([FILE IT](#)) evitam a maioria dos enganos antes de qualquer coisa ser escrita. Se você ativou o log de auditoria, digite [Undo last filing](#). Caso contrário, feche ou edite o ticket no seu rastreador normalmente.

Ele pode me ajudar a verificar ou fechar um bug?

Sim. Peça um comentário de verificação e ele redige uma nota clara de aprovado/reprovado (nos seus idiomas) no ticket existente, e você aprova antes de ser postado.

Posso usar para mais de um projeto?

Cada pasta do BugIt é configurada para o rastreador de um projeto. Para outro projeto, rode o [Begin setup](#) de novo para apontá-lo para outro lugar, ou mantenha uma cópia separada por projeto.

Como eu recebo atualizações?

Quando uma nova versão sai, o BugIt menciona isso quando você inicia um chat. Digite [update](#) e ele instala no local: seu glossário, estilo da casa e configurações são mantidos, e ele faz backup deles primeiro.

O que acontece quando minha licença expira?

Quando o seu período termina, renove a partir da sua conta; o seu dispositivo aplica o direito renovado na próxima verificação online. A licença offline de 72 horas é separada; cobre apenas uma indisponibilidade temporária do servidor de licenciamento, não um período expirado. Digite [License status](#) a qualquer momento.

Se eu renovar ou comprar outra licença, os dias se acumulam?

Não, as licenças não se acumulam. Uma renovação **substitui** o seu direito atual; o prazo começa do zero e os dias não usados não são transferidos, então renove perto do fim do prazo.

PARTE 6 · LICENÇA E SUPORTE

Termos de licença (resumo em linguagem simples)

Os termos completos e vinculantes estão no arquivo [LICENSE](#) na sua pasta do BugIt, e esse documento é quem manda. Em resumo:

TÓPICO	EM PALAVRAS SIMPLES
Licenciado, não vendido	Você compra uma licença para usar o BugIt, não a propriedade dele. A Taskivator mantém todos os direitos.
Seus assentos	Solo = um dispositivo por vez; Team = até 5 membros. Sua conta e o seu direito são pessoais; não compartilhe, revenda nem publique.
Revogação	Um acesso compartilhado, reembolsado, contestado (chargeback) ou usado indevidamente pode ser revogado.
Personalize, mas...	Edite seu config, glossário e templates livremente, mas não desative nem contorne o licenciamento/ativação.
Sua responsabilidade	Como você usa o BugIt e a segurança do seu projeto são de sua responsabilidade. Você revisa e aprova todo relatório antes de ser registrado. As proteções são auxílios, não garantias.
"Como está", responsabilidade	Fornecido "como está", sem garantia. No limite que a lei permitir, a responsabilidade é limitada ao valor que você pagou, excluindo danos indiretos/consequentes.
Prazo e legislação	Uma licença de um ano que começa no dia em que você ativa (uma compra única, sem renovação automática), regida pelas leis do Japão. Reembolsos são tratados pela loja onde você comprou.
Renovações não se acumulam	Uma renovação substitui o seu direito atual: o prazo começa do zero e os dias não usados não são transferidos, então renove perto do fim do prazo.

Dois documentos na sua pasta

[LICENSE](#) (termos completos) e [PRIVACY.md](#) (declaração de privacidade). Ao ativar e usar o BugIt, você aceita a LICENSE.

Como obter ajuda

Por favor, confira primeiro a Solução de problemas e o FAQ acima; eles cobrem quase tudo. Depois, nesta ordem:

1 · Peça para a IA corrigir

Seu melhor primeiro passo: descreva o problema no chat e peça para o assistente corrigir. Se ele mudar um arquivo e parecer certo, clique em Keep para aplicar (só no seu PC).

2 · Rode uma verificação de saúde

`Check status` · `Check readiness` · ou `python tools/selftest.py` no Terminal.

3 · Restaure

`Restore my settings` desfaz uma mudança ou atualização ruim.

4 · Fale com um humano

Só se o guia e a IA não conseguirem ajudar: visite bugit.dev ou escreva para support@bugit.dev (o suporte é feito em inglês). Problema na configuração nos seus primeiros 7 dias? A gente te deixa funcionando ou ajuda com um reembolso pela loja.

Por favor, mantenha informações confidenciais do seu projeto fora dos e-mails de suporte

Cada projeto é diferente, e boa parte do seu trabalho pode ser confidencial. Se você nos escrever, descreva o problema do BugIt apenas em termos gerais, e não cole código confidencial, detalhes de bugs, especificações, screenshots ou dados do seu projeto. A Taskivator não se responsabiliza por nenhuma informação confidencial enviada a nós, e qualquer coisa confidencial que recebermos é excluída imediatamente. Sempre que possível, peça ajuda ao assistente de IA primeiro; ele pode resolver a maioria dos problemas direto no seu editor.

Obrigado por escolher o BugIt.

Registre tickets impecáveis, pule as duplicatas e recupere seu tempo, um bug de cada vez.

BugIt QA Agent · Guia de Configuração e Uso · © 2026 Taskivator. All Rights Reserved. · bugit.dev · Os arquivos LICENSE e PRIVACY.md incluídos no seu download são os documentos que regem o uso.